NCPD Threat Detection System: Jetson Orin Nano Deployment Plan

This document outlines the step-by-step process for deploying the Cyberpunk NCPD Threat Detection system onto your NVIDIA Jetson Orin Nano (15W) using the `dusty-nv/jetson-inference` library.

The Approach

While you could theoretically edit the default `detectnet.py` script provided in the `jetson-inference` repository, `detectnet.py` is primarily designed to output to a local display window (using OpenGL) or standard video files.

To achieve the **exact same Cyberpunk web interface** we built, the best approach is to use the custom Flask application (`app.py`) we just created. This script already imports `jetson.inference` and `jetson.utils` and serves the MJPEG video stream along with the Cyberpunk HTML/CSS directly to a web browser.

Here is the step-by-step plan to get it running on your Jetson Orin Nano.

Phase 1: Hardware & Environment Setup on Jetson

1. **Boot the Jetson Orin Nano**: Ensure it is powered (15W mode is recommended for running `ssd-mobilenet-v2` smoothly) and connected to your network.
2. **Connect the USB Camera**: Plug your USB camera into one of the Jetson's USB ports. It will typically mount as `/dev/video0`.
3. **Install Jetpack & jetson-inference**:

   - Ensure NVIDIA JetPack is installed.
   - Follow dusty-nv's guide to clone and build the `jetson-inference` repository:

     ```
     git clone --recursive https://github.com/dusty-nv/jetson-infere
     cd jetson-inference
     mkdir build && cd build
     cmake ../
     make -j$(nproc)
     sudo make install
     sudo ldconfig
     ```

   - During the cmake step, ensure you download the `ssd-mobilenet-v2` model.

Phase 2: Transferring the Project

1. **Copy the Files**: Transfer the project folder we just built on Windows over to the Jetson. You can use `scp`, a USB drive, or clone it via Git. You need the following structure:

   ```
   Antigravity/
   â”œâ”€â”€ app.py
   ```

```
â”œâ”€â”€ static/
â”,   â””â”€â”€ style.css
â””â”€â”€ templates/
    â””â”€â”€ index.html
```

2. **Install Python Web Dependencies**:

    - On the Jetson, open a terminal and navigate to the `Antigravity` folder.
    - Install Flask and OpenCV (OpenCV should already be installed with JetPack, but Flask is needed):

      ```
      pip3 install Flask
      ```

Phase 3: Running the Application

1. **Execute the Script**:

    - Run the application using Python 3. The script is already programmed to detect `HAS_JETSON = True` automatically when it successfully imports the jetson libraries.

      ```
      python3 app.py
      ```

2. **Access the Interface**:

    - Open a web browser on a computer connected to the same network as the Jetson.
    - Navigate to `http://<JETSON_IP_ADDRESS>:5000` (e.g., `http://192.168.1.50:5000`).

Alternative Approach: Modifying `detectnet.py` Directly

If your goal is strictly to edit `detectnet.py` without using Flask, you can modify it to use Jetson's built-in WebRTC server instead of a local display. However, **you will lose the custom Cyberpunk HTML/CSS layout**.

Here is how you would edit `detectnet.py` safely by creating a copy:

1. **Locate the Script**: Navigate to the examples directory in the jetson-inference repository.

   ```
   cd jetson-inference/python/examples/
   ```

2. **Create a Copy**: To avoid modifying the original file provided by dusty-nv, create a copy named `anomaly_detectnet.py`.

   ```
   cp detectnet.py anomaly_detectnet.py
   ```

3. **Modify the Output String**: Open `anomaly_detectnet.py` in your preferred editor. You can change the output argument when calling the script to point to the WebRTC server, or modify the code directly to handle custom rendering.

   ```
   # Instead of outputting to display://0
   python3 anomaly_detectnet.py /dev/video0 webrtc://@:8554/output
   ```

4. **View the Stream**: You can then open your browser and navigate to `http://<JETSON_IP_ADDRESS>:8554` to view the raw WebRTC stream.

Conclusion: For the full NCPD Cyberpunk experience with the custom sci-fi HUD bounding boxes, stick to Phase 1-3 using the `app.py` file we created. It seamlessly bridges Jetson Inference's raw power with standard web development UI capabilities.